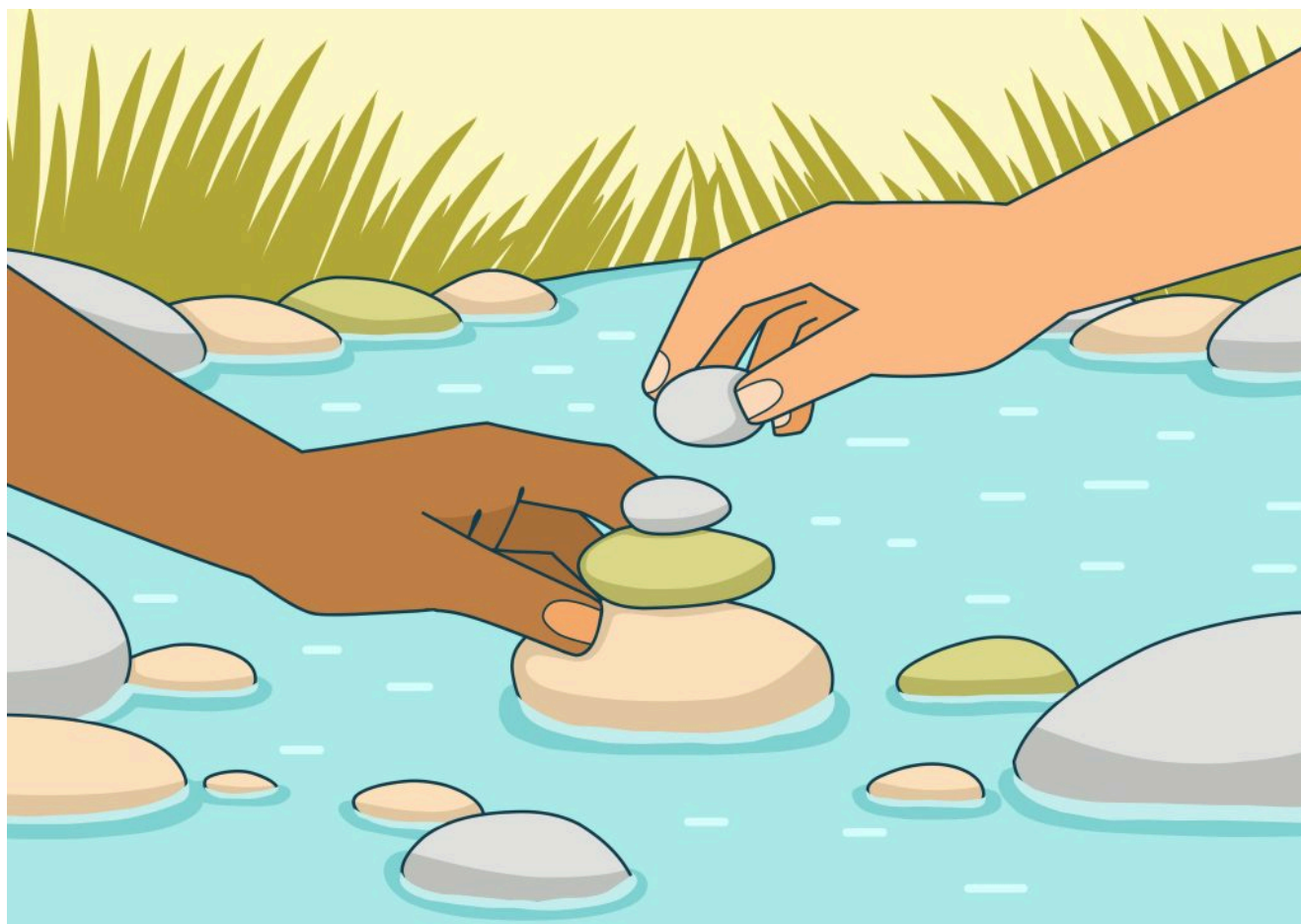


ДАННЫЕ / БАЗЫ ДАННЫХ / С ОТКРЫТЫМ ИСХОДНЫМ КОДОМ

В течение многих лет Apache Cassandra перекладывала эту работу на вашу команду, но в версии 6.0 мы возвращаемся к ней

В альфа-версии Apache Cassandra 6.0 добавлены ACID-транзакции Accord, метаданные транзакционного кластера и функция восстановления, благодаря которой операции выполняются в базе данных.

8 июня 2026 года, 10:00 [Анил Инамдар](#)



Милхад для Unsplash+

[NetApp](#) спонсировала этот пост.

Выпуски Apache Cassandra, как правило, оцениваются примерно на тех же уровнях. Что он может сделать сейчас, чего не мог раньше? На версию 6.0,

связаны с новыми возможностями. Скорее, тема для версии 6.0 – это почтенная база данных NoSQL, возвращающая работу, которую она всегда неявно просила выполнить кого-то другого.

Я тесно сотрудничаю с коммиттерами Cassandra и **два года назад** писал о Cassandra 5.0 на этих (виртуальных) страницах. Этот релиз предоставил командам несколько потрясающих новых инструментов, таких как **собственная векторная индексация** и индексы с подключением к хранилищу. Он укрепил позиции Cassandra в сфере ИИ-нагрузок. Мы воочию убедились в прогрессе: возможности проекта в области ИИ используются в производственных средах во многих кластерах, которыми мы управляем в NetApp Instaclustr. Но я считаю, что в версии 6.0 аргументы в пользу использования Cassandra звучат по-другому, и, на мой взгляд, с этим различием стоит разобраться до того, как команды начнут планировать пути обновления.

Проблема координации

Любой, кто создавал многосекционные рабочие процессы в Cassandra, знает, в чем заключается основная сложность. Она всегда связана с кодом приложения, потому что именно там всегда происходила координация многосекционных процессов. База данных не определяет, каким должно быть допустимое состояние, что должно происходить, если из-за частичного сбоя работа прерывается на полпути, и как обеспечить согласованность при записи данных в кластер.

«С практической точки зрения часть этой работы по координации теперь переносится из кода приложения в саму базу данных».

Accord в Cassandra 6.0 — это протокол консенсуса без лидера, который обеспечивает семантику **транзакций ACID** со строгой сериализуемой изоляцией между несколькими разделами. С практической точки зрения часть работы по координации теперь переносится из кода приложения в саму базу данных. Текущая реализация требует тщательного планирования схемы

⇒ заменяет существующую логику приложения. Однако командам,

The metadata problem

Transactional Cluster Metadata (TCM) may be the most consequential change in this release. I would also bet it gets the least attention, because “improved cluster metadata coordination” does not make for an especially click-happy headline.



Spot by NetApp enables cloud operations teams to deliver scalability, performance and security for cloud infrastructure and applications — at the lowest possible cost — through continuous automation and optimization combined with deep visibility and governance. Learn more at spot.io.

[Learn More →](#)

Cassandra has always propagated metadata through Gossip Protocol and eventual consistency. Token ownership and schema changes (as well as data placement decisions) were all spread across nodes over time without strong ordering guarantees. While a functional model, certain classes of failure are genuinely hard to diagnose, and certain operations hard to sequence predictably. (Anyone who has untangled a messy Cassandra cluster that got into a bad state during a schema migration or topology change can now mentally relive the experience I am describing.)

TCM offers a Cluster Metadata Service to maintain a tidy log of state transitions, replacing Gossip as the primary mechanism for this work. Gossip stays in the picture but is no longer doing the heavy lifting. Your end result is a cluster whose operators can see what changed and when, and operations that have historically behaved unpredictably will now be more deterministic.

TRENDING STORIES

1. [The database storage problem is solved. Here's what comes next.](#)
2. [When your data model is the bottleneck: lessons from Medium's feature store](#)

5. For years, Apache Cassandra handed this work to your team — 6.0 takes it back

Repair, which is no longer someone else's problem

Our team at NetApp Instaclustr has managed repair orchestration on behalf of customers for many a year. The approaches vary, from cron jobs to custom scripts to third-party tooling of different quality levels and beyond. The underlying reality, however, does not. Repair is how Cassandra maintains replica consistency over time; it has to happen, and *until now* Cassandra itself had nothing to say about how or when.

“Repair is how Cassandra maintains replica consistency over time. Until now Cassandra itself had nothing to say about how or when.”

I have watched teams under-run repair and pay for it gradually. I have also watched teams run it too aggressively and pay for it all at once. Neither outcome is exactly Cassandra's fault, because the tooling to do it right has always existed. The challenge was that it lived outside the database, which meant it lived outside what the database could operationally guarantee. With 6.0, full and incremental repair is now live inside Cassandra, with scheduling controls and treatment as an operational concern rather than something bolted on from the outside looking in. A parallel to **Unified Compaction Strategy in 5.0**, you have another background process that used to require external infrastructure now within the database's own responsibility. Teams running their own clusters without dedicated Cassandra expertise on staff are the ones most likely to feel this on day one.

Also pay attention to...

Three other changes are worth looking at before you finalize any upgrade plan, including the constraints framework. This update brings column-level validation directly into the schema, covering range checks, null constraints, regex patterns, and JSON validation, so that logic does not have to be replicated across every write or read. Application-level validation still has its place; this just adds an important layer of defense closer to the data itself.

uniform the data is, and managing the dictionary lifecycle will inevitably add some operational overhead to factor in. I think most organizations with high-volume, pattern-heavy workloads will find the tradeoff worthwhile after testing. Others may reasonably decide otherwise.

New cursor-based compaction processes SSTable data more incrementally, reducing heap pressure and GC overhead during compaction. Operators will not configure this directly. For workloads with heavy compaction activity, the effect should manifest as more stable node behavior under sustained load.

What I'm watching

I have been working with Cassandra clusters at NetApp Instaclustr since 2019. During that time, I have watched the project mature through two major releases and seen firsthand which **operational gaps** kept recurring in the clusters we manage. Version 6.0 addresses several of the ones that came up most often.

The features in this release share a theme that I find more interesting than any one of them individually. The project is taking ownership of operational work it has long delegated elsewhere. Coordination logic that lived in application code, metadata consistency managed through eventual propagation, and replica reconciliation handed off to external tooling are all challenges the Cassandra community has been working toward solving for years. Version 6.0 is where that work lands.

The maturity here is different than expanding what the database can store or index, as its the database acknowledging that work it has been handing off to ops teams and application developers is properly its own work to do. Whether that plays out the way it should depends largely on how Accord holds up as real production workloads start hitting it. That's the part of 6.0 I'll be watching most closely. But of course, as always, test against your actual workloads before upgrading anything that matters. The best upgrades are the ones you understand before they hit production.

TNS



Anil Inamdar is the global head of Data Services at NetApp Instaclustr, which provides a managed platform around open source data technologies including Cassandra, Kafka, Postgres, ClickHouse and OpenSearch. Anil has more than 20 years of experience in data and...

SHARE THIS STORY

TRENDING STORIES

1. The database storage problem is solved. Here's what comes next.
2. When your data model is the bottleneck: lessons from Medium's feature store
3. Why MotherDuck refuses to fork DuckDB
4. Why the need for humans won't disappear in the age of autonomous databases
5. For years, Apache Cassandra handed this work to your team — 6.0 takes it back

INSIGHTS FROM OUR SPONSOR

Spot by NetApp enables cloud operations teams to deliver scalability, performance and security for cloud infrastructure and applications — at the lowest possible cost — through continuous automation and optimization combined with deep visibility and governance. Learn more at spot.io.

[Learn More →](#)

most recent TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter

THE NEW STACK

[About / Contact](#)
[Sponsors](#)
[Advertise With Us](#)
[Contributions](#)



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

[Frontend Developer Roadmap](#)
[Backend Developer Roadmap](#)
[Devops Roadmap](#)

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)
[Terms of Use](#)
[Advertising Terms & Conditions](#)
[Privacy Policy](#)
[Cookie Policy](#)